

```

• PlayerInputComponent->BindAction("Action", IE _
Pressed, this, &AUnrealCPPCharacter::OnAction);
•
• // Bind movement events
• PlayerInputComponent->BindAxis("MoveForward", this,
&AUnrealCPPCharacter::MoveForward);
• PlayerInputComponent->BindAxis("MoveRight", this,
&AUnrealCPPCharacter::MoveRight);
•
• // We have 2 versions of the rotation bindings to
handle different kinds of devices differently
• // "turn" handles devices that provide an absolute
delta, such as a mouse.
• // "turnrate" is for devices that we choose to
treat as a rate of change, such as an analog joystick
• PlayerInputComponent->BindAxis("Turn", this,
&APawn::AddControllerYawInput);
• PlayerInputComponent->BindAxis("TurnRate", this,
&AUnrealCPPCharacter::TurnAtRate);
• PlayerInputComponent->BindAxis("LookUp", this, &APaw
n::AddControllerPitchInput);
• PlayerInputComponent->BindAxis("LookUpRate", this,
&AUnrealCPPCharacter::LookUpAtRate);
• }
•
• void AUnrealCPPCharacter::OnFire()
• {
• // try and fire a projectile
• if (ProjectileClass != NULL)
• {
• UWorld* const World = GetWorld();
• if (World != NULL)
• {
• const FRotator SpawnRotation = Get-
ControlRotation();
• // MuzzleOffset is in camera space,
so transform it to world space before offsetting from the
character location to find the final muzzle position
• const FVector SpawnLocation =
((FP_MuzzleLocation != nullptr) ? FP_MuzzleLocation-
>GetComponentLocation() : GetActorLocation()) + SpawnRota-

```